

torch.linspace#

<https://pytorch.org/docs/stable/generated/torch.linspace.html>

Description:

Creates a one-dimensional tensor of size steps whose values are evenly spaced from start to end, inclusive. That is, the value are: From PyTorch 1.11 linspace requires the steps argument. Use steps=100 to restore the previous behavior. start (float or Tensor) – the starting value for the set of points. If Tensor, it must be 0-dimensional end (float or Tensor) – the ending value for the set of points. If Tensor, it must be 0-dimensional steps (int) – size of the constructed tensor

Function Signature

```
torch.linspace(start, end, steps, *, out=None, dtype=None, layout=torch.strided, device=None, requires_grad=False) → Tensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

out (Tensor, optional) – the output tensor. dtype (torch.dtype, optional) – the data type to perform the computation in. Default: if None, uses the global default dtype (see `torch.get_default_dtype()`) when both start and end are real, and corresponding complex dtype when either is complex. layout (torch.layout, optional) – the desired layout of returned Tensor. Default: `torch.strided`. device (torch.device, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see `torch.set_default_device()`). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types. requires_grad (bool, optional) – If autograd should record operations on the returned tensor. Default: False.

Code Examples

```
>>> torch.linspace(3, 10, steps=5)
tensor([ 3.0000,  4.7500,  6.5000,  8.2500, 10.0000])
>>> torch.linspace(-10, 10, steps=5)
tensor([-10.,  -5.,   0.,   5.,  10.])
>>> torch.linspace(start=-10, end=10, steps=5)
tensor([-10.,  -5.,   0.,   5.,  10.])
>>> torch.linspace(start=-10, end=10, steps=1)
tensor([-10.])
```

Detailed Documentation

Documentation

Creates a one-dimensional tensor of size `steps` whose values are evenly spaced from `start` to `end`, inclusive. That is, the value are:

From PyTorch 1.11 `linspace` requires the `steps` argument. Use `steps=100` to restore the previous behavior.

`start` (float or Tensor) – the starting value for the set of points. If Tensor, it must be 0-dimensional

`end` (float or Tensor) – the ending value for the set of points. If Tensor, it must be 0-dimensional

`steps` (int) – size of the constructed tensor

`out` (Tensor, optional) – the output tensor.

`dtype` (torch.dtype, optional) – the data type to perform the computation in. Default: if None, uses the global default dtype (see `torch.get_default_dtype()`) when both `start` and `end` are real, and corresponding complex dtype when either is complex.

`layout` (torch.layout, optional) – the desired layout of returned Tensor. Default: `torch.strided`.

`device` (torch.device, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see `torch.set_default_device()`). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types.

`requires_grad` (bool, optional) – If autograd should record operations on the returned tensor. Default: False.